

UCS503 - SOFTWARE ENGINEERING

Launder Lens

GNN-Based Money Laundering Detection

Submitted by:

1024170308 Suryansh Malik
1024170309 Tanmay Agarwal
1024170310 Sparsh Kumar
1024170422 Divakar Purohit

Group: 7 BE Third Year, CSE

Submitted to:

Dr. Reaya Grewal

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	4
1.1	Higher-order Goal	4
1.2	Background and Context	4
1.3	Problem Statement	5
1.4	Project Objectives	5
1.5	Time-to-Value	6
2	Proposed Solution	7
2.1	Overview	7
2.2	Core Workflow	7
2.3	Operational Constraints	8
2.4	Project Scope and Deliverables	8
2.4.1	Initial Deliverable	8
2.4.2	Subsequent Deliverables	9
3	Methodology and Design	10
3.1	Solution Approach	10
3.1.1	Duplicate/Quality Assistance	10
3.2	System Architecture	10
3.2.1	High-Level Design	11
3.2.2	Web Architecture	11
3.2.3	Technical Stack	11
3.3	Implementation Details	11
3.3.1	Module 1: Core Functionality	12
3.3.2	Module 2: Secondary Features	12
3.4	CI/CD and Fast Delivery Enabler	12
3.5	Scalability	13
3.6	Engine Availability Heuristic	13
4	Evaluation and Testing	14
4.1	Evaluation Criteria	14
4.2	Primary Evaluation Metric	14
4.3	Secondary Evaluation Metrics	14
4.4	Testing Methodology	15
4.5	Pilot Validation Plan	15
4.6	Performance Metrics	15
4.7	Analysis	16

5	Risks, Conclusion and Future Work	17
5.1	Risks and Mitigations	17
5.2	Summary of Work	17
5.3	Key Findings	18
5.4	Future Work and Recommendations	18
5.5	Final Remarks	18
5.6	Summary	19
	Preliminary Feasibility Study	20
	Technical Feasibility	20
	Managerial Feasibility	20
	Economic Feasibility	20
	Financial Feasibility	20
	Cultural Feasibility	21
	Social Feasibility	21
	Safety Feasibility	21
	Political Feasibility	21
	Preliminary Decision	21
6	UML Diagrams	22
6.1	Use Case Scenario	22
6.2	Use Case Diagram	24
6.3	Class Diagram	25
6.4	Activity Diagram	26
6.5	Links	27
	Bibliography	28

Introduction

1.1 Higher-order Goal

The higher-order goal of LaunderLens is to improve Anti-Money Laundering (AML) transaction monitoring through machine learning and graph-based analysis. The system aims to identify potentially suspicious financial transactions and provide useful information to AML analysts for further investigation.

Money laundering activities may involve multiple interconnected accounts and a sequence of transactions rather than a single suspicious transaction. Therefore, LaunderLens examines financial activity from both transaction-level and relational perspectives. Traditional machine-learning models analyze individual transaction characteristics, while the proposed Graph Neural Network (GNN) represents accounts as nodes and transactions as directed edges to capture relationships within the transaction network.

The proposed system is intended to support human analysts by generating suspicious-transaction predictions, risk scores, and connected account/transaction information rather than independently making final AML decisions.

1.2 Background and Context

Anti-Money Laundering refers to the processes used by financial institutions to identify and investigate potentially suspicious financial activity. Transaction monitoring is an important part of AML because financial institutions must process large numbers of transactions and identify activities that may require further investigation.

Automated detection is challenging because laundering transactions represent a small proportion of overall financial activity, resulting in a highly imbalanced classification problem. Furthermore, suspicious activity may not always be identifiable from the characteristics of an individual transaction. Multiple accounts can be used to transfer funds through chains, intermediary accounts, layering structures, and fan-in or fan-out patterns.

LaunderLens uses IBM's HI-Small synthetic AML dataset for development and evaluation. The primary transaction file, `HI-Small_Trans.csv`, contains transaction information including timestamp, sending and receiving accounts, transaction amounts, currencies, payment format, and the `Is Laundering` label. The dataset also contains `HI-Small_accounts.csv` and `HI-Small_Patterns.txt`, with the latter providing information about generated laundering patterns.

The project first applies transaction-level machine-learning models, including Decision Tree, Random Forest, XGBoost, and SVM, as experimental baselines. It then extends the analysis to a graph-based representation in which accounts are represented as nodes and transactions are represented as directed edges.

1.3 Problem Statement

Traditional machine-learning approaches for AML detection primarily represent each transaction using its individual attributes, such as transaction amount, currency, payment format, and time. Although these characteristics can provide useful information for detecting suspicious transactions, they do not explicitly represent the relationships between accounts.

Money laundering can involve several interconnected accounts through transaction chains and other network structures. For example, a sequence such as $A \rightarrow B \rightarrow C \rightarrow D$ may contain information that is not represented when each transaction is considered independently.

The problem addressed by LaunderLens is therefore to develop and evaluate a graph-based AML detection approach that can incorporate relationships between accounts and transactions. The project compares transaction-level machine-learning models with a GNN-based edge-classification approach without deliberately weakening the traditional ML models or providing them with graph-derived features.

1.4 Project Objectives

The objectives of LaunderLens are:

1. To preprocess and analyze the IBM HI-Small synthetic AML transaction dataset.
2. To develop transaction-level AML detection baselines using Decision Tree, Random Forest, XGBoost, and SVM.
3. To evaluate the developed models using Precision, Recall, F1-score, PR-AUC, ROC-AUC, and Confusion Matrix.
4. To construct a transaction graph in which bank accounts are represented as nodes and transactions are represented as directed edges.
5. To develop a Graph Neural Network for transaction-level edge classification using transaction attributes and account relationships.
6. To compare transaction-level and graph-based representations for AML detection.

7. To visualize selected multi-account laundering patterns using NetworkX and Gephi.
8. To develop the system toward an analyst-oriented AML monitoring workflow involving suspicious-transaction detection, risk scoring, network investigation, and AML reporting.

1.5 Time-to-Value

LaunderLens follows an incremental development approach in which useful components are implemented and evaluated before progressing to more complex components. The initial stage focused on understanding the IBM HI-Small dataset and preparing transaction-level features.

The next stage involved implementing classical machine-learning baselines using Decision Tree, Random Forest, XGBoost, and SVM. These models established the transaction-level detection baseline and provided the initial evaluation results.

The project subsequently moved toward graph analysis using NetworkX and Gephi. An initial transaction network was generated and examined, leading to the decision to focus visualization on smaller, meaningful laundering patterns rather than the complete dense transaction graph.

The current stage is the transition from classical machine learning to Graph Neural Network implementation. The next development increment involves constructing the account-transaction graph, implementing a GCN or GraphSAGE model, performing edge classification, and evaluating the results against the completed transaction-level baselines.

Proposed Solution

2.1 Overview

LaunderLens is an AI-based AML transaction monitoring system designed to detect potentially suspicious transactions and assist analysts in investigating their surrounding financial activity.

The proposed system receives transaction data from a financial institution and performs data validation and preprocessing before constructing an account-transaction graph. In this graph, each bank account is represented as a node and each transaction is represented as a directed edge between the sender and receiver accounts.

A Graph Neural Network processes the graph and predicts whether individual transaction edges are potentially associated with laundering activity. The system can then generate risk scores and display suspicious transactions to an AML analyst. The analyst can examine the connected account and transaction network, investigate the case, record the case outcome, and generate an AML report.

The current implementation has completed the transaction-level machine-learning baseline stage and initial graph visualization work. The GNN-based detection component is currently being developed.

2.2 Core Workflow

The proposed LaunderLens workflow begins when a financial institution provides transaction data to the system. The system validates the input and preprocesses the transaction records by extracting relevant numerical, categorical, and temporal features.

The processed transactions are then used to construct an account-transaction graph. Each account becomes a graph node, while every transaction forms a directed edge from the sending account to the receiving account. Transaction attributes are stored as edge features, while **Is Laundering** represents the edge-level target label.

The GNN processes the graph and performs edge classification to identify potentially suspicious transactions. The system generates risk scores for the detected transactions and presents the results to the AML analyst.

The analyst can select a suspicious transaction, inspect its connected accounts and surrounding transaction network, investigate the case, and mark the case as suspicious or legitimate. An AML report can subsequently be generated from the investigation information.

2.3 Operational Constraints

The project uses the IBM HI-Small synthetic AML dataset rather than real financial institution data. This allows the system to be developed without exposing real customer or financial information.

The large volume of transaction data introduces computational constraints, particularly for graph construction and GNN training. The project therefore requires efficient preprocessing and graph representation. The SVM experiment uses LinearSVC on a subset of the data because applying an expensive kernel-based SVM to the complete transaction volume would be computationally demanding.

The dataset is highly imbalanced, with laundering transactions representing a small portion of the total transactions. Consequently, model evaluation focuses on Precision, Recall, F1-score, PR-AUC, ROC-AUC, and Confusion Matrix rather than relying primarily on accuracy.

The GNN implementation begins with a relatively simple architecture such as GCN or GraphSAGE to maintain an appropriate scope for the semester project.

2.4 Project Scope and Deliverables

The project covers transaction-level AML detection, graph construction, graph-based machine learning, transaction-network visualization, and the development of an analyst-oriented AML monitoring workflow.

2.4.1 Initial Deliverable

The initial deliverable includes the acquisition and understanding of the IBM HI-Small dataset, preprocessing of transaction data, feature extraction, and development of classical machine-learning baselines.

Decision Tree, Random Forest, XGBoost, and SVM have been implemented using transaction-level features including transaction amounts, currencies, payment format, hour, day, and day of week. A chronological 60% training, 20% validation, and 20% testing strategy is being used.

The initial evaluation framework includes Precision, Recall, F1-score, PR-AUC, ROC-AUC, and Confusion Matrix.

Initial graph visualization work has also been performed using NetworkX and Gephi. A GEXF transaction graph has been generated and examined, with further work focused on creating smaller visualizations based on known laundering patterns.

2.4.2 Subsequent Deliverables

The subsequent deliverables include construction of the complete account-transaction graph and implementation of the GNN-based edge-classification model.

The GNN will use transaction attributes as edge features and **Is Laundering** as the edge label. The completed GNN will be evaluated using the same primary classification metrics as the classical baselines.

Further development will include focused visualization of laundering patterns, risk-score presentation, suspicious transaction investigation, an analyst-oriented dashboard, case management, and AML report generation.

Methodology and Design

3.1 Solution Approach

LaunderLens uses two complementary machine-learning approaches. The first approach consists of transaction-level classical ML models that analyze individual transaction characteristics. Decision Tree, Random Forest, XGBoost, and SVM are used as experimental baselines.

The second approach uses graph-based machine learning. The transaction dataset is transformed into a directed graph where accounts are nodes and transactions are edges. A GNN is then used to learn representations from the graph and classify individual transaction edges.

The comparison is intended to examine the difference between transaction-level and relational representations. The classical ML models are not intentionally weakened, and graph-derived features such as account degree, centrality, or PageRank are not added to their input features.

3.1.1 Duplicate/Quality Assistance

The system performs transaction-data quality checks during preprocessing to ensure that required fields are available and correctly formatted. Timestamp information is parsed to obtain temporal features, while categorical transaction attributes are encoded for machine-learning models.

Graph-based analysis also provides assistance in identifying relationships between accounts and transactions. Connected transactions can be examined to identify chains and other structures associated with known laundering patterns. Such information is intended to assist investigation rather than automatically determine that an account or transaction represents financial crime.

3.2 System Architecture

The proposed architecture consists of transaction-data input, preprocessing, graph construction, machine-learning detection, risk assessment, analyst investigation, network visualization, and reporting components.

The classical ML models form the experimental transaction-level baseline branch, while the GNN forms the main graph-based detection component of the proposed system.

3.2.1 High-Level Design

The high-level system flow is:

**Financial Institution → Transaction Data → Data Validation →
Preprocessing → Transaction Graph Construction → GNN → Suspicious
Transaction Detection → Risk Score → AML Analyst → Network
Investigation → AML Report**

The current project has implemented the dataset processing and classical ML stages and has started graph visualization. The GNN and complete analyst-facing system remain under development.

3.2.2 Web Architecture

The proposed application follows a three-tier architecture consisting of a frontend, backend/API layer, and data layer.

The frontend will provide interfaces for transaction submission, detection results, suspicious-case investigation, and network visualization. The backend will handle authentication, data processing, model inference, risk scoring, and case-related operations. The database will store relevant users, transactions, model results, case information, and report data.

The ML and graph-processing components will operate within the backend processing pipeline.

3.2.3 Technical Stack

The project uses Python for implementation and Pandas and NumPy for data processing. Scikit-learn is used for classical machine-learning models, while XGBoost is used for gradient-boosting-based classification.

PyTorch and PyTorch Geometric are being used for the GNN implementation. NetworkX is used for graph processing and Gephi is used for transaction-network visualization.

The IBM HI-Small synthetic AML dataset serves as the primary dataset. React, FastAPI, and PostgreSQL form the proposed application stack for the complete system.

3.3 Implementation Details

The implementation is divided into transaction-level detection and graph-based detection components.

3.3.1 Module 1: Core Functionality

The first module implements transaction-level AML detection. Transaction records are loaded from `HI-Small-Trans.csv` and relevant features are extracted, including Amount Received, Amount Paid, Receiving Currency, Payment Currency, Payment Format, Hour, Day, and Day of Week.

Categorical features are one-hot encoded before being provided to the machine-learning models.

Decision Tree, Random Forest, XGBoost, and SVM have been implemented as baseline models. The data is divided chronologically into 60% training, 20% validation, and 20% testing data.

The models are evaluated using Precision, Recall, F1-score, PR-AUC, ROC-AUC, and Confusion Matrix due to the highly imbalanced nature of the dataset.

3.3.2 Module 2: Secondary Features

The second module focuses on graph-based AML detection.

Every unique bank account is assigned a node identifier. Each transaction is represented as a directed edge from the sender account to the receiver account.

The graph representation is:

Node = Bank Account
Edge = Transaction

Transaction attributes such as Amount Received, Amount Paid, Receiving Currency, Payment Currency, Payment Format, and temporal information are used as edge features.

The `Is Laundering` field is used as the edge-level label. Therefore, the primary GNN task is edge classification.

The initial implementation will use GCN or GraphSAGE. NetworkX and Gephi are also being used to visualize selected transaction structures and known laundering patterns.

3.4 CI/CD and Fast Delivery Enabler

CI/CD is intended to provide a repeatable development and deployment process for LaunderLens. Automated builds, tests, and linting can be executed whenever changes are introduced into the project.

The planned pipeline will support reproducible testing of preprocessing and application components and allow validated versions of the system to be deployed to a staging environment.

At the current stage, CI/CD is part of the planned engineering workflow rather than a completed core component.

3.5 Scalability

AML transaction-monitoring systems must be capable of processing large numbers of transactions and accounts. LaunderLens therefore considers scalability at both the data and graph levels.

Efficient preprocessing, indexed data access, sparse graph representations, and modular separation between frontend, backend, and ML components can support larger datasets.

For graph visualization, the complete transaction network may become too dense for meaningful analysis. Therefore, focused subgraphs based on selected suspicious transactions or known laundering patterns can be extracted for visualization.

The proposed architecture also allows the ML and application components to be scaled independently as system requirements increase.

3.6 Engine Availability Heuristic

LaunderLens can be developed using established open-source technologies. Pandas and NumPy provide the required data-processing functionality, while Scikit-learn and XGBoost provide established classical ML implementations.

PyTorch and PyTorch Geometric provide the required deep-learning and GNN functionality. NetworkX and Gephi provide graph-processing and visualization capabilities.

The availability of these mature tools allows the project to focus on AML detection methodology, graph representation, model evaluation, and system integration rather than developing underlying infrastructure from scratch.

Evaluation and Testing

4.1 Evaluation Criteria

The models are evaluated as binary classifiers for identifying potentially laundering transactions. Since laundering transactions represent a small proportion of the dataset, evaluation emphasizes metrics that provide meaningful information about minority-class detection.

The primary evaluation measures are Precision, Recall, F1-score, PR-AUC, and ROC-AUC. Confusion matrices are also used to examine the distribution of correct and incorrect predictions.

Accuracy is not treated as the primary metric because a highly imbalanced dataset can produce high accuracy even when the minority laundering class is poorly detected.

4.2 Primary Evaluation Metric

F1-score is used as the primary summary metric because it combines Precision and Recall into a single measure.

For AML detection, Precision represents the proportion of transactions predicted as suspicious that are actually laundering transactions, while Recall represents the proportion of actual laundering transactions that are detected by the model.

F1-score provides a combined view of these two aspects and is therefore used alongside the other evaluation metrics.

4.3 Secondary Evaluation Metrics

Precision is used to measure the correctness of suspicious-transaction predictions.

Recall is used to measure the ability of the model to identify actual laundering transactions.

PR-AUC is used to evaluate model performance across classification thresholds while focusing on the precision-recall relationship, which is particularly relevant for imbalanced data.

ROC-AUC is used to evaluate the model's ability to distinguish between laundering and non-laundering transactions across thresholds.

Confusion matrices are used to examine True Positives, False Positives, True Negatives, and False Negatives.

4.4 Testing Methodology

Testing is performed at multiple levels. Individual preprocessing operations are checked to ensure that transaction fields are correctly parsed, encoded, and transformed.

The machine-learning pipeline is tested from data loading through feature extraction, model training, prediction, and evaluation.

The chronological data split is used to separate training, validation, and testing transactions while maintaining their temporal order.

For the graph pipeline, account identifiers, edge directions, edge features, and edge labels are verified to ensure that transactions are correctly represented in the graph. Integration testing will subsequently verify the complete flow from transaction data to graph construction, GNN prediction, and risk-score generation.

4.5 Pilot Validation Plan

The system will be validated using the held-out portion of the IBM HI-Small dataset. Classical ML models will be evaluated using the established evaluation metrics, followed by evaluation of the GNN after its implementation.

The laundering patterns described in `HI-Small_Patterns.txt` will be used to identify representative multi-account structures for graph analysis. Selected patterns will be visualized using Gephi to examine how suspicious transactions are connected through intermediary accounts.

The final system can additionally be validated through an analyst-oriented workflow in which a suspicious transaction is selected, its connected network is examined, and the case is recorded for reporting.

4.6 Performance Metrics

The performance comparison will contain the results of the completed Decision Tree, Random Forest, XGBoost, and SVM experiments using Precision, Recall, F1-score, PR-AUC, and ROC-AUC.

The GNN will be evaluated using the same metrics after its implementation and training.

The GNN results will be added after completion of the current implementation stage.

Model	Precision	Recall	F1	PR-AUC	ROC-AUC
Decision Tree	0.925466	0.143961	0.249164	0.157530	0.920153
XGBoost	0.014443	0.842512	0.028400	0.231372	0.933463
SVM	0.0079	0.8783	0.0157	0.0141	0.9221

Table 4.1: AML Model Performance Comparison

4.7 Analysis

The completed classical ML models provide a transaction-level baseline for AML detection. Their results show how effectively individual transaction attributes can be used to distinguish laundering from non-laundering transactions.

The graph-based approach addresses a different aspect of the problem. Instead of considering a transaction independently, the GNN operates on a network containing connected accounts and transactions.

For example, the transaction $A \rightarrow B$ can be represented within a larger structure such as $A \rightarrow B \rightarrow C \rightarrow D$. This allows the GNN to incorporate information from neighboring accounts and transactions.

The final comparison will therefore examine whether the additional relational information represented by the graph contributes useful information for detecting transactions involved in multi-account laundering patterns. The project does not assume beforehand that the GNN will outperform every classical ML model.

Risks, Conclusion and Future Work

5.1 Risks and Mitigations

The highly imbalanced nature of the dataset can make model evaluation difficult. This is addressed by emphasizing Precision, Recall, F1-score, PR-AUC, ROC-AUC, and Confusion Matrix rather than accuracy alone.

The large number of transactions can create computational challenges during pre-processing, SVM experimentation, and graph construction. Efficient processing, appropriate data subsets where necessary, and GPU-based computation can help manage these limitations.

The complete transaction graph can become extremely dense and difficult to visualize. Focused subgraphs and selected known laundering patterns are therefore used for meaningful visualization.

False-positive predictions represent another important limitation because a flagged transaction does not necessarily establish that money laundering has occurred. The proposed system therefore keeps human AML analysts involved in investigation and case decisions.

The complexity of GNN development is managed by beginning with a relatively simple GCN or GraphSAGE architecture before considering more advanced graph models.

5.2 Summary of Work

LaunderLens has progressed through several stages of development. The IBM HI-Small synthetic AML dataset has been obtained and analyzed, and transaction-level features have been prepared for machine-learning experiments.

Decision Tree, Random Forest, XGBoost, and SVM have been implemented as transaction-level baseline models. A chronological 60% training, 20% validation, and 20% testing strategy has been adopted, with evaluation based on Precision, Recall, F1-score, PR-AUC, ROC-AUC, and Confusion Matrix.

Graph-based analysis has also been initiated using NetworkX and Gephi. An initial transaction graph has been generated, and further visualization is being focused on known laundering patterns.

The current development stage is the transition to GNN-based edge classification, where accounts will be represented as nodes and transactions as directed edges.

5.3 Key Findings

The project has established transaction-level machine-learning models as a baseline for AML detection.

The highly imbalanced nature of the dataset demonstrates the importance of using Precision, Recall, F1-score, PR-AUC, and ROC-AUC for evaluation.

The transaction graph representation provides a way to explicitly represent relationships between accounts and transactions. Initial graph visualization also demonstrates that large transaction networks can contain complex interconnected structures.

These observations motivate the implementation of a GNN to investigate whether relational information can provide useful additional information for detecting multi-account laundering activity.

5.4 Future Work and Recommendations

The immediate future work is to complete construction of the account-transaction graph and implement the initial GCN or GraphSAGE model for edge classification.

After implementation, the GNN will be trained and evaluated using the same evaluation framework as the classical ML models. The results will then be compared with the transaction-level baselines.

Further work will include focused visualization of known laundering patterns in Gephi, integration of GNN predictions with risk scores, and development of an analyst-oriented interface.

The proposed final system can include suspicious-transaction dashboards, connected-account network visualization, case investigation, case-status management, and AML report generation.

Advanced future extensions may include temporal GNN architectures, attention-based graph models, improved model explainability, and larger-scale deployment.

5.5 Final Remarks

LaunderLens approaches AML transaction monitoring using both transaction-level and graph-based machine learning. The completed classical ML models provide a baseline based on individual transaction characteristics, while the proposed GNN approach represents the relationships between accounts and transactions.

The central focus of the project is to investigate the usefulness of relational information for detecting suspicious transactions that may form part of larger multi-account laundering structures. The current project has completed the classical ML stage and initial graph visualization work and is progressing toward GNN-based edge classification.

5.6 Summary

LaunderLens currently consists of a completed transaction-level AML machine-learning baseline, initial transaction-network visualization, and a proposed graph-based detection component.

The completed work includes the IBM HI-Small dataset, preprocessing, Decision Tree, Random Forest, XGBoost, SVM, evaluation methodology, and initial Gephi analysis.

The current development focus is GNN implementation, followed by GNN evaluation, comparison with classical ML models, and integration into the proposed AML analyst workflow.

Preliminary Feasibility Study

The preliminary feasibility study for LaunderLens evaluated the project across technical, managerial, economic, financial, cultural, social, safety, and political dimensions. The original feasibility assessment concluded that no major blockers had been identified and recommended proceeding to detailed scoping, requirement gathering, and system design.

Technical Feasibility

The project can be implemented using Python and established open-source machine-learning and graph-processing libraries. The use of a synthetic AML dataset avoids access requirements associated with real financial data. The project is technically feasible at course-project scale, with detailed implementation choices evolving as the GNN and application components are developed.

Managerial Feasibility

The project is being developed by a four-person team. The work can be divided among data and detection, graph/GNN development, application development, and system integration and documentation. The semester duration provides a manageable development period when the project scope is controlled and development is carried out incrementally.

Economic Feasibility

The primary investment for the project is development time rather than direct financial expenditure. The project provides practical experience in machine learning, graph neural networks, data processing, visualization, and software engineering, making the expected learning value appropriate for an academic project.

Financial Feasibility

The project primarily relies on free and open-source software libraries and publicly available synthetic data. Therefore, no significant direct monetary cost is expected for the current academic implementation.

Cultural Feasibility

AML is an internationally relevant financial-compliance domain. The project uses synthetic transaction data and focuses on software-based detection and investigation, so no significant cultural constraints have been identified for the academic implementation.

Social Feasibility

The proposed system is designed to assist AML analysts rather than independently make final decisions. Suspicious-transaction predictions are intended to support investigation, while the analyst remains responsible for reviewing connected transaction activity and recording the case outcome.

Safety Feasibility

The project is software-only and uses synthetic financial transaction data. As a result, it does not introduce physical safety risks and has limited privacy concerns. Basic authentication and access control are considered for the proposed application.

Political Feasibility

The project is an academic AML software-engineering exercise using synthetic transaction information. It does not process real financial records or implement regulatory decision-making, so no significant political or regulatory barrier has been identified for the current project scope.

Preliminary Decision

The preliminary feasibility assessment supports proceeding with the project. The current development has progressed from the initial proposal into dataset analysis, classical ML experimentation, graph visualization, and preparation for GNN implementation.

UML Diagrams

This chapter presents the UML representations of the LaunderLens AML Transaction Monitoring and Money Laundering Detection System.

6.1 Use Case Scenario

The primary use case of the LaunderLens system is the detection and investigation of suspicious transactions. The workflow begins when the Financial Institution or Bank uploads transaction data. The system validates and preprocesses the data, constructs an account-transaction graph, and uses the GNN model to detect suspicious transactions and generate risk scores.

The AML Analyst or Investigator can view the detection results, select suspicious transactions, inspect the connected account and transaction network, investigate the identified activity, and mark the case as suspicious or legitimate. The system can subsequently generate an AML report.

The System Administrator manages system configuration, users and access permissions, system monitoring, and GNN model updates or retraining.



Figure 6.1: Use Case Scenario of the LaunderLens AML Detection System

6.2 Use Case Diagram

The Use Case Diagram illustrates the interactions between the major actors and the functional components of the LaunderLens system. The primary actors are the Financial Institution/Bank, AML Analyst/Investigator, and System Administrator.

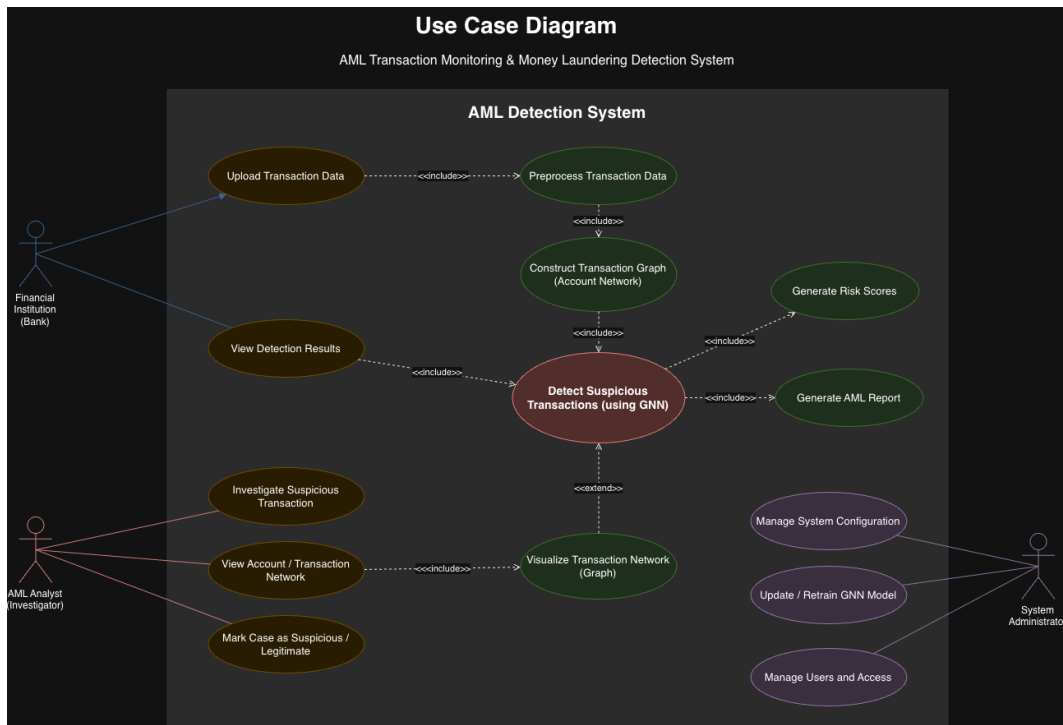


Figure 6.2: Use Case Diagram of the LaunderLens AML Detection System

6.3 Class Diagram

The Class Diagram represents the structural organization of the LaunderLens system, including the major entities involved in transaction processing, account representation, detection results, risk scoring, and system management.

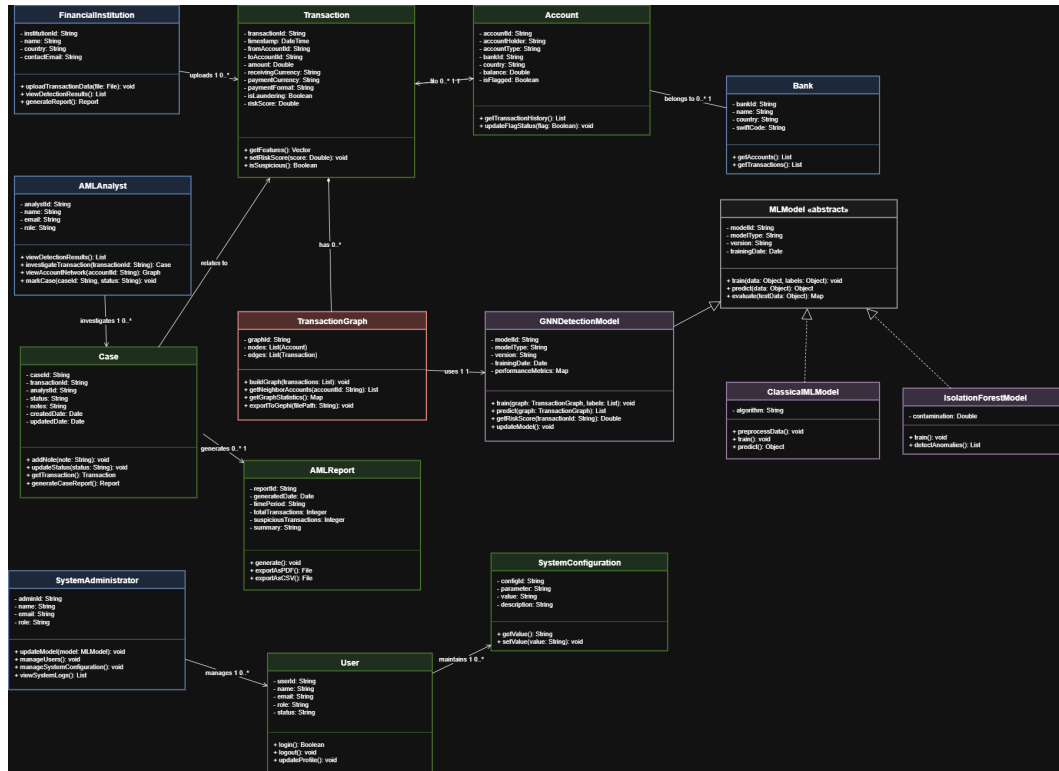


Figure 6.3: Class Diagram of the LaunderLens AML Detection System

6.4 Activity Diagram

The Activity Diagram represents the end-to-end workflow of the LaunderLens system, beginning with transaction data upload and preprocessing, followed by graph construction, GNN-based detection, risk-score generation, and analyst investigation.

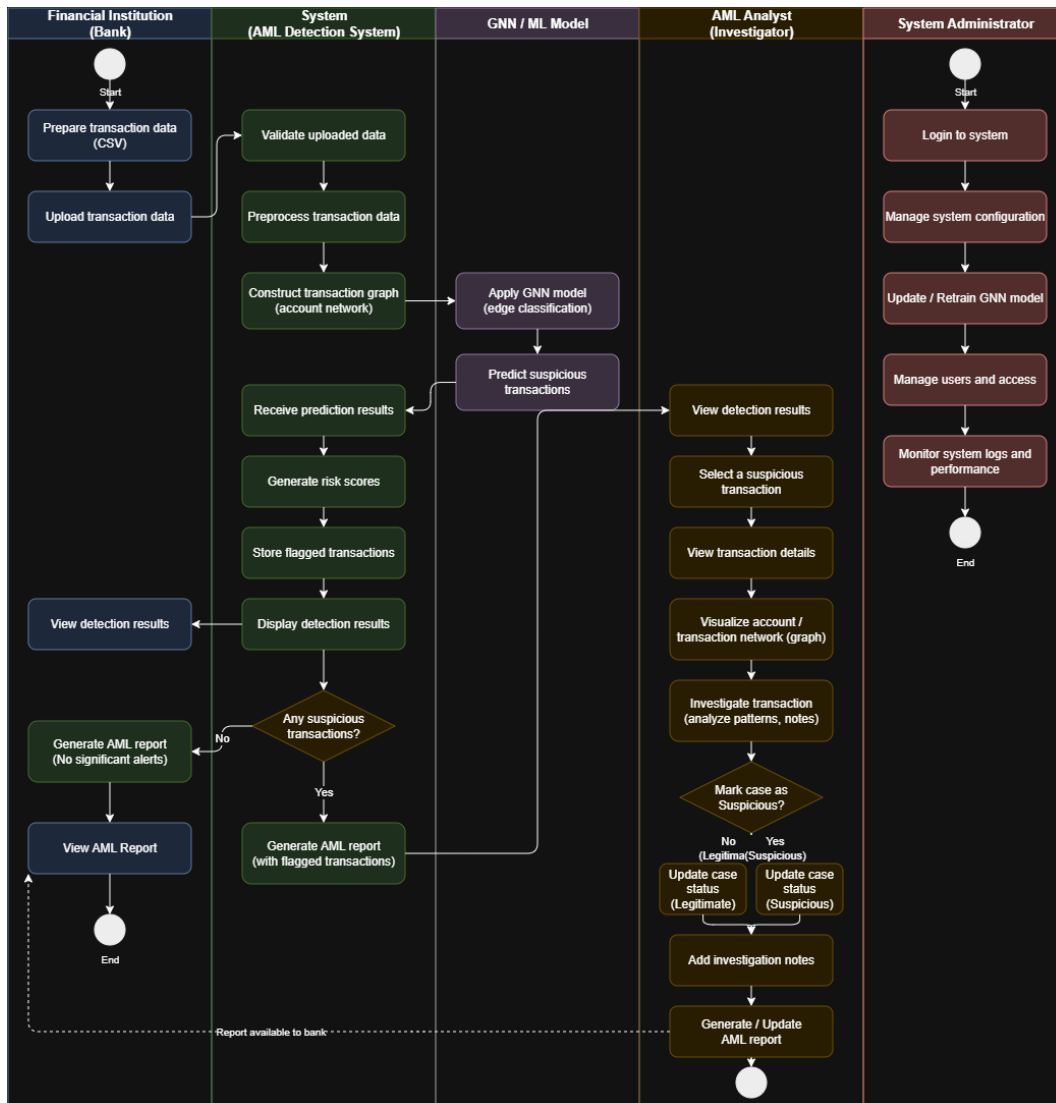


Figure 6.4: Activity Diagram of the LaunderLens AML Detection Workflow

6.5 Links

The following link provides access to the UML diagrams of the LaunderLens AML Detection System:

- **UML Diagrams:** <https://github.com/Launder-Lens/LaunderLens-lite/tree/main/reports/uml>

Bibliography

- [Alt26] Erik Altman. *IBM Transactions for Anti Money Laundering (AML)*. Kaggle dataset. 2026. URL: <https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>.
- [Dee20] DeepFindr. *Understanding Graph Neural Networks*. YouTube video series. 2020. URL: <https://www.youtube.com/watch?v=f0ctJB4kV1M&list=PLV8yxwG0xvvoNkzPfCx2i8an--Tkt708Z>.
- [GKT+22] Lucas Schmidt Goecks, André Luis Korzenowski, Platão Gonçalves Terra Neto, Davenilcio Luiz de Souza, and Taciana Mareth. “Anti-money laundering and financial fraud detection: A systematic literature review”. In: *Intelligent Systems in Accounting, Finance and Management* 29.2 (2022), pp. 71–85.